`x/usdcmigration`: Security Review

Scope: `disperze/usdc-migration@2f93210` (`master`, release line v10): `x/usdcmigration/**` (module, keeper, ICS20 middleware, IBC v2 middleware), `proto/usdcmigration/v1/*`, plus the documented wiring in `README.md` and `docs/integration.md`. `testing/simapp` was used as the reference app. Out of scope: the `release/v8.x` line, each chain's real `app.go`, the off-chain repayment, and upstream Cosmos SDK, ibc-go and CometBFT (except where noted).

Track: B (Cosmos SDK v0.53.x / ibc-go v10.x). Pinned: `cosmos-sdk v0.53.4`, `ibc-go/v10 v10.5.0`, `cometbft v0.38.17`, `cosmos/gogoproto v1.7.0`, Go 1.24.

Date: 2026-10-28.

Method: the `cosmos-ibc-audit` workflow: recon (`detect_versions.py`, `app_wiring.py`), threat model, mechanical scan (`scan.py`), the ABCI/determinism, SDK-module, IBC-apps/middleware and known-vulnerability checklists, then PoC tests on a three-chain `ibctesting` setup with the repo's simapp. `govulncheck` was also run.

Summary

Severity	Count	Confirmed	Likely	Needs investigation
Critical	0			
High	0			
Medium	1	1		
Low	1		1	
Info	8	2	6	

The module is small, has no block-level code and no module account, and keeps all logic on tx-level paths where a panic only fails that tx. The rules work as described: the receive block, the send block on IBC v1 and v2 (gov included), and returns that move no funds on this chain. I found no unprivileged path to mint, drain, halt or bypass the blocks.

The most important fix is **M-01**. A return packet is recognised by comparing its ack or timeout with the *current* params. A gov param change while a return is in flight therefore makes the ICS20 app mint unbacked voucher to the gov account. The PoC confirms it: supply went from 1,000,000 to 2,000,000. The runbook warns about this, but recording pending returns in state removes the hazard for a few lines of code. Since the module is unreleased, that change needs no migration.

One candidate looked like a chain halt and was disproved by its end-to-end PoC: the middleware decodes acks without a canonical check. It is downgraded to **I-01**, because ibc-go core already rejects those acks before any app callback runs.

Unpatched dependencies

No pinned version is exposed to an advisory in the skill's Cosmos table (ASA-2025-004 and ISA-2025-001 are fixed in ibc-go v10.1.0, and the distribution overflow in SDK v0.53.3). This module is a library, so its `go.mod` only sets *minimum* versions; each chain's own `go.mod` and Go toolchain decide what ships. See I-08.

Component	Pinned (floor)	Advisory / fix	Fixed in	Exposure through this module
cometbft	v0.38.17	GO-2026-4361 (commit-signature vs block-time inconsistency)	v0.38.21	none in module code; chain-level
ibc-go	v10.5.0	non-UTF-8 strings in events (v10.5.1); callbacks `OnTimeoutPacket` blocked by unmarshal error (v10.6.0); transfer v2 validation parity (v10.7.0)	v10.5.1 / v10.6.0 / v10.7.0	none in module code; v10.6.0 matters to chains with callbacks (Xion, Coreum)
cosmos-sdk	v0.53.4	malformed-tx panic hardening (non-advisory)	v0.53.8	none in module code
Go stdlib, x/net, grpc, otel	go1.24.7, x/net v0.41.0, grpc v1.75.0	52 reachable advisories reported by `govulncheck`, all in the toolchain or node networking stack	various	none in module logic

System overview

- **Module:** `x/usdcmigration`. State is one `collections.Item[Params]`. No module account, no permissions, no Begin/End/PreBlocker, no hooks, no upgrade handler (each chain writes its own).
- **Msgs:** `MsgUpdateParams` and `MsgReturnToNoble`. Both use signer `authority` and are checked with `checkAuthority` in the msg server.
- **IBC v1:** `usdcmigration.IBCMiddleware` wraps the ICS20 stack outermost. Its `ICS4Wrapper` is the keeper.
- `OnRecvPacket` checks the receive block.
- `OnAcknowledgementPacket` and `OnTimeoutPacket` swallow the error ack or timeout of a return.
- Everything else is passed through.
- **ICS4:** `Keeper` is the transfer keeper's ICS4 wrapper. `SendPacket` applies the send block, then forwards to the next wrapper (PFM, rate limit, callbacks or the channel keeper). `MsgReturnToNoble` writes its packet directly to that next wrapper.
- **IBC v2:** `v2.IBCMiddleware` wraps the transfer v2 route. `OnSendPacket` applies the send block; receive, ack and timeout pass through.
- **Reference app (`testing/simapp`):** gov is removed from the bank blocked addresses (simapp default). The transfer module holds `Minter` / `Burner` permissions. There are no ABCI++ handlers, CosmWasm or unordered txs.

Threat model

Actor	Controls	Relevant to
Any account, ICA host account, contract, EVM precompile caller	ICS20 sends of the voucher (direct, `authz`, PFM forward, ibc-hooks, `IbcMsg::Transfer`)	send block
Relayers (permissionless)	whether and when packets, acks and timeouts are delivered; cannot forge ack bytes	return timeouts, refund path
Noble (counterparty chain)	packet data of incoming transfers; the ack bytes for returns, bound by the light client	receive block, ack decoding
Other counterparty chains	packets on non-Noble channels; can open new channels (permissionless)	receive block scope
Governance (authority)	params, returns (amount, receiver, memo, timeout), bank freeze, ordering of proposals	M-01, I-02, I-04

Actor	Controls	Relevant to
Chain integrators	ICS4 and middleware wiring	L-01

Attack-surface table

#	Entry point	Path(s)	Caller trust	Ante	Gas metered	State written	Funds moved
1	`MsgUpdateParams`	gov proposal	gov	no (gov EndBlocker)	no (gov EndBlocker)	`Params`	none
2	`MsgReturnToNoble`	gov proposal	gov	no	no	packet commitment (via next ICS4)	none here; releases Noble's escrow
3	`Keeper.SendPacket`	transfer keeper: `MsgTransfer`, `authz` `MsgExec`, ICA host, PFM, ibc-hooks, wasm, precompile	any	direct only	yes	none (reject or forward)	none
4	`v2.OnSendPacket`	`MsgTransfer` over a v2 client, `MsgSendPacket`	any	yes	yes	none	none
5	`OnRecvPacket`	relayer `MsgRecvPacket`	counterparty	yes (relayer tx)	yes	none (error ack)	none
6	`OnAcknowledgementPacket`	relayer `MsgAcknowledgement`	counterparty ack bytes	yes	yes	event only, for returns	prevents the refund mint
7	`OnTimeoutPacket`	relayer `MsgTimeout`, `MsgTimeoutOnClose`	relayer (timing)	yes	yes	event only, for returns	prevents the refund mint
8	`Query/Params`	gRPC/REST	anyone	n/a	query	none	none
9	`InitGenesis` / `ExportGenesis`	genesis, `RunMigrations`	n/a	n/a	n/a	`Params`	none

Findings

[M-01] A param change while a return is in flight mints unbacked voucher to gov

Severity	Medium. It is a governance footgun with a large blast radius (rubric). It would be Low if you treat the runbook rule "do not submit `MsgUpdateParams` until the return settles" as a sufficient control.
Status	Confirmed (PoC executed)
Component	`x/usdcmigration` keeper and v1 middleware

Location	`x/usdcmigration/keeper/return.go:42-46` , `x/usdcmigration/ibc_middleware.go:55-87`
Track / versions	SDK v0.53.4, ibc-go v10.5.0
Attacker	governance (a mistake, or two proposals whose execution overlaps)
Class	mint (unbacked supply) / accounting

Description

The module stores nothing about the returns it sends. On an ack or timeout, `IsReturnPacket` decides whether a packet was a return by checking it against the *current* params:

```
// keeper/return.go:42-46
params := k.GetParams(ctx)
return sender == k.authority && params.IsNobleChannel(sourceChannel) &&
    denomPath == types.VoucherPath(sourceChannel, params.BaseDenom)
```

If `noble_channels` or `base_denom` changed after the return was sent, an error ack or timeout of that return is no longer recognised. The middleware then passes it to the ICS20 app. The return burned nothing, but the app treats the packet as a gov transfer of a voucher it burned. It refunds by minting the voucher and sending it to gov with keeper-level `SendCoins` (`ibc-go/v10 modules/apps/transfer/keeper/relay.go:276-286`). That send does not check blocked addresses, so the mint succeeds even on chains where gov is still a blocked address.

The design has a second dependency on mutable state. Correct classification also relies on `send_block_enabled` staying `true` . With it `false` , an ordinary gov `MsgTransfer` of the voucher on a Noble channel burns real voucher. If that transfer gets an error ack or times out, the middleware swallows it as a "return", and gov is never refunded.

Impact

- Voucher supply grows by up to the full return amount, and none of it is backed by Noble's escrow.
- The README takes the holder snapshot at the height of the `usdcmigration_return_to_noble` event. After a refunded return, the retry happens at a later height, by which time gov holds the minted voucher. A snapshot at the retry height then counts gov as a holder of `amount` , diluting real holders by up to 50% if the first return was for the whole escrow.

Preconditions and likelihood

A gov proposal that changes params executes between the return's execution and its settlement. For example, a "wind down: clear `noble_channels` " proposal is already in voting when the return runs, and Noble answers with an error ack or the packet times out. The runbook warns against this, which lowers the likelihood, but the warning is the only control.

Proof of concept

`TestPoC_M01_ParamChangeInFlightMintsVoucherToGov` in `x/usdcmigration/audit_poc_test.go` (build tag `auditpoc`):

```
go test -tags auditpoc ./x/usdcmigration/ -run 'TestMiddlewareTestSuite/TestPoC_M01' -v
```

Observed: `voucher supply 1000000 -> 2000000, gov voucher balance 1000000`. The test fails on the supply assertion.

Recommendation

Record each return when it is sent, and match acks and timeouts by packet identity instead of by params:

```
// keys.go
var PendingReturnsKey = collections.NewPrefix(1)

// keeper.go
PendingReturns collections.KeySet[collections.Pair[string, uint64]] // (source channel, sequence)

// return.go, after a successful SendPacket
err = k.PendingReturns.Set(ctx, collections.Join(msg.ChannelId, sequence))

// IsReturnPacket(ctx, sourceChannel, sequence): PendingReturns.Has(...)
// middleware: on any ack or timeout of a pending return, Remove the key first.
// On an error ack or timeout, emit usdcmigration_return_refunded and return nil.
// On a success ack, pass it through as today.
```

This makes the params safe to change at any time and drops the runbook rule. It also stops gov's ordinary transfers from being misclassified when `send_block_enabled=false`. State stays bounded: one key per gov return, removed on settlement. The module is unreleased (`CHANGELOG.md`: first release `v10.0.0`), so no migration is needed. Port the same change to `release/v8.x`.

References: `ibc-apps-middleware.md` §9 (per-packet state cleaned up on ack/timeout; undo exactly what send did, once).

[L-01] The send block depends on wiring the module cannot check, and it fails open

Severity	Low
Status	Likely (traced in the docs and ibc-go code; no chain-specific PoC)
Component	integration: `docs/integration.md`, `Keeper` as the transfer keeper's ICS4 wrapper
Location	`x/usdcmigration/keeper/ics4_wrapper.go:25-40`; `docs/integration.md` "Rules that matter"
Attacker	any account, once the wiring is wrong
Class	control bypass

Description

The v1 send block only runs if *every* copy of the transfer keeper sends through this keeper. The ibc-go transfer keeper is a value type, and some chains re-point it after construction. Two of the target chains do so today: Xion and Coreum call `app.TransferKeeper.WithICS4Wrapper(cbStack)` (see `docs/integration.md`). PFM, the wasm keeper and other modules hold earlier copies of the keeper.

If any copy keeps a wrapper that bypasses `UsdcMigrationKeeper`, transfers through it are not blocked. Nothing errors, and nothing at runtime shows the gap. The same applies if the v2 transfer route is not wrapped on a chain that has one, or if a chain's ICS20 port is not `transfer`. In that last case, `Keeper.SendPacket` and `returnPacketData` quietly skip every packet.

Impact

Holders can move the voucher out after the upgrade:

- To Noble: this is a legitimate redemption, but it shrinks the escrow below the `amount` already in the gov proposal. The return then gets an error ack, and gov needs another voting period close to the 2027-01-12 deadline.
- To other chains: the voucher escapes the freeze and the snapshot.

No funds are stolen.

Recommendation

- Ship a reusable wiring check and make each chain run it in its app tests. One helper is enough: it executes a voucher `MsgTransfer` through the app's `MsgServiceRouter`, and through PFM's and wasm's transfer paths, in a `CacheContext`, and asserts `ErrSendBlocked`. On chains with a v2 route, also run it over a v2 client.
- Optionally run the same check at the end of the upgrade handler and log the result. Don't fail the handler: a failed upgrade handler halts the chain.

Informational

[I-01] The middleware decodes return acks without the canonical-bytes check. Confirmed at module level; blocked end to end by ibc-go core. `ibc_middleware.go:57-58` decodes the ack with `transfertypes.ModuleCdc.UnmarshalJSON` and branches on `!ack.Success()`. gogoproto's jsonpb fills `oneof` members by ranging over a Go map (`gogoproto@v1.7.0/jsonpb/jsonpb.go:1104`), so an ack carrying both `result` and `error` decodes either way at random.

Called directly with identical input, the middleware takes the "refund" branch in most runs and passes the ack to the transfer app, which rejects it, in the rest. Two runs of 200 gave 179/21 and 169/31. That divergence would fork the chain. It is not reachable today: ibc-go core's `AcknowledgePacket` re-marshals every ack and rejects non-canonical bytes before any app callback (`ibc-go/v10@v10.5.0 modules/core/04-channel/keeper/packet.go:385-392`, the ISA-2025-001 fix; the same check is in v8.7.0 at `packet.go:433-438`). The end-to-end test, with Noble committing the bytes and a relayer submitting them with a valid proof, is rejected 12 of 12 times with `acknowledgement marshalling error`.

Recommendation: mirror the transfer app's check, so safety doesn't depend on the core version floor:

```
if err := transfertypes.ModuleCdc.UnmarshalJSON(acknowledgement, &ack); err == nil &&
    bytes.Equal(transfertypes.ModuleCdc.MustMarshalJSON(&ack), acknowledgement) && !ack.Success() {
```

PoCs: `TestPoC_I01_MiddlewareAckDecodeIsNonDeterministic` fails as expected; `TestMitigation_I01_CoreRejectsAmbiguousAck` passes.

[I-02] The default return timeout (1 h) is short for a packet sent by gov. The packet is created in the gov EndBlocker at the end of the voting period. If no relayer delivers it within an hour, it times out, and the retry costs another voting period. *Recommendation:* default to 24 h (the README example already uses `86400`) or make `timeout_seconds` required. Before submitting, also check that the Noble client on this chain is active and not near expiry, otherwise `SendPacket` fails and the proposal fails.

[I-03] Params are not checked against chain state. `Params.Validate` checks only the `channel-N` format. A typo, or a channel that isn't Noble's transfer channel, leaves the module inert or aimed at the wrong channel, with no error. The README lists three chains whose voucher hash had to be computed by hand. *Recommendation:* in `UpdateParams`, and in a helper that upgrade handlers call, reject channels for which `ics4Wrapper.GetAppVersion(ctx, "transfer", ch)` returns `!ok`. In each chain's upgrade test, assert that the voucher denom exists.

[I-04] Refunding a return skips every middleware below this one, and the memo is free-form. A return's error ack or timeout is answered without calling the wrapped stack. Rate-limit "undo send" accounting (Neutron, or Stride-style limits) and callbacks therefore never see it. The docs say to leave the memo empty, but nothing enforces it: a callbacks key or a PFM memo would trigger behaviour in wrappers below the keeper. *Recommendation:* reject a non-empty `memo` in `Msg ReturnToNoble.ValidateBasic` (or drop the field), and state the rate-limit interaction in the integration notes.

[I-05] Snapshot and repayment caveats from the bank freeze. These are off-chain but follow from the design. - `MsgSetSendEnabled(false)` stops `MsgSend` and ICS20 sends (`relay.go:70`), but not ICS20 *unescrow* on receive (`relay.go:314-315` uses `SendCoins`) or module-account flows such as DEXes. Holders of multi-hop vouchers on other chains can still bring them back after the snapshot. - The holder set at the snapshot height includes this chain's ICS20 escrow addresses for those channels, plus module and contract accounts. Funds paid to an escrow address are stranded.

Recommendation: in the repayment plan, exclude or specially map escrow addresses and module accounts.

[I-06] IBC v2 receives are not checked. Safe today, because a v2 voucher's trace uses the destination *client* ID, so it is a different denom from `transfer/channel-N/uusdc`. ibc-go v10.5.0 also has no v1-channel aliasing (`04-channel/v2/keeper/keeper.go:31` keeps `channelKeeperV1`, but nothing uses it). *Recommendation:* re-check if a future ibc-go adds aliasing. Adding the same receive check to `v2.IBCMiddleware.OnRecvPacket` is cheap.

[I-07] The receive block emits no event. `OnRecvPacket` returns `NewErrorAcknowledgement`, which carries only `ABCI code: 3` with no codespace, and returns before the transfer app emits its `fungible_token_packet` event. Noble users and explorers get an ambiguous reason. *Recommendation:* emit an `usdcmigration_recv_blocked` event (channel, denom, sequence) on that path.

[I-08] Dependency floors. See "Unpatched dependencies" above. *Recommendation:* in the README, list *recommended* patch levels next to the minimums: ibc-go ≥ v10.6.0 for chains with callbacks, the latest cometbft v0.38.x (≥ v0.38.21), and a patched Go toolchain for release builds.

Checklist coverage

Reference	Applied	Notes
`version-notes.md` Track B	yes	no capability module; IBC v2 route coverage checked
`abci-determinism.md`	yes	no block-level code; JSON decoding checked → I-01
`cosmos-sdk-checklist.md` §1-2, 4, 7, 10-12	yes	authority, alternative paths (authz, ICA host, wasm, PFM, gov), bank and supply, genesis round trip, amino names
`cosmos-sdk-checklist.md` §3, 5, 6, 8, 9	n/a	no ante decorators, math, hand-built keys, hooks or staking code in the module
`ibc-core-checklist.md`	partially	no core or light-client code; core ack validation read for I-01
`ibc-apps-middleware.md` §1, 2, 9	yes	stack composition, ICS4 symmetry, v1 vs v2, ICS20 accounting, refunds
`ibc-apps-middleware.md` §3-8	wiring only	ICA, callbacks, hooks, PFM and rate limits checked only for how they interact with returns
`known-vulns.md`	yes	ASA-2025-004 / ISA-2025-001 pattern → I-01; denom-path spoofing pattern checked (no finding)
Actual chain `app.go` files	no	out of scope; see L-01

Appendix

Candidates triaged and dismissed

- `scan.py` **PANIC hits** (`NewKeeper` , `GetParams` , `InitGenesis` , `RegisterGRPCGatewayRoutes` , simapp): they run at init time, or inside a tx where baseapp recovers the panic. The module has no Begin/End/PreBlocker, so none can halt the chain. gov runs proposal msgs with panic recovery.
- `scan.py` **DECODE hits:** the middleware and keeper decode packet data with the same `Fungible TokenPacketData.UnmarshalJSON` the transfer app uses (strict, `DisallowUnknownFields` ; `transfer/types/encoding.go`), and the v2 middleware calls the same `UnmarshalPacketData` . The block and the app cannot disagree about the denom. Failing open on decode errors is safe, because the wrapped app then fails on the same input.
- `scan.py` **NONDET hit** (`range app.keys`): only in the test app.
- **Denom-path spoofing on send:**
- v1 `MsgTransfer` resolves `ibc/HASH` to the full path before building the packet.
- transfer v2 rejects base denoms containing `/` (`transfer/v2/ibc_module.go:72`).
- A native denom literally named `transfer/channel-N/uusdc` still produces that packet string, so it is blocked too.
- Moving the real voucher coin needs the exact SHA-256 preimage path.
- **Other ways to unescrow Noble's escrow:** only a packet committed on `transfer/channel-N` with the voucher path does that. Sends on the `transfer` port all go through the transfer keeper (blocked) or this keeper's return. Wasm contracts can only send on their own ports.
- **Receive-block scope:** only the base denom arriving on a configured channel mints the target voucher (the `ReceiverChainIsSource` logic). Other Noble denoms and multi-hop paths create other denoms.
- **Authorization:** both msgs check `msg.Authority == k.authority` in the msg server. authz, ICA host and wasm dispatch cannot satisfy the check unless gov itself grants it.
- **Gas and growth:** one params read per ICS20 packet, and no module-owned state that users can grow.

PoC tests

File: `x/usdcmigration/audit_poc_test.go` (build tag `auditpoc`, so it does not run in CI).

```
go test -tags auditpoc ./x/usdcmigration/ -run 'TestMiddlewareTestSuite/Test(PoC|Mitigation)' -v
```

Test	Finding	Result on `2f93210`
`TestPoC_M01_ParamChangeInFlightMintsVoucherToGov`	M-01	FAIL (as expected): supply 1,000,000 → 2,000,000
`TestPoC_I01_MiddlewareAckDecodeIsNonDeterministic`	I-01	FAIL (as expected): both outcomes for identical input (for example 179/21, 169/31 of 200)
`TestMitigation_I01_CoreRejectsAmbiguousAck`	I-01	PASS: core rejects 12/12 ambiguous acks

The existing suite (`go test ./...`) passes unchanged.